

Cómo funcionan los agentes de IA de esta app (resumen de una hoja)

24 de septiembre de 2026

Explicación corta del flujo, pensada para presentarla a alguien que no ha visto el código. Si necesitas el detalle completo (todas las herramientas, cómo agregar una nueva, límites y pruebas), eso está en `docs/ia-herramientas.md`; este documento es el resumen de entrada.

La idea en una frase

El modelo de IA nunca calcula. Solo decide *qué herramienta usar y con qué datos*. Los números siempre salen de los mismos motores de cálculo (`js/calc/*.js`) que usan las calculadoras normales de la app — la IA nunca "hace la cuenta" ni de memoria ni por su cuenta.

Esto es la técnica de **function calling**: la app le describe al modelo un catálogo de funciones disponibles ("herramientas"); el modelo responde algo como "*quiero llamar a `calcular_perdidas` con estos datos*"; la app la ejecuta de verdad y le devuelve el resultado exacto para que lo interprete y redacte la respuesta.

El flujo, paso a paso

1. El usuario escribe una pregunta en el chat

▼

2. El modelo (Gemini / OpenAI / Claude, según lo configurado) la lee junto con el historial de la conversación y la lista de herramientas que ese agente puede usar

— Puede responder directo con texto (no necesitó calcular nada)

— Puede pedir ejecutar una o varias herramientas

▼

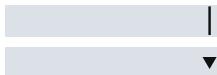
3. La app valida los datos que pidió el modelo (tipos, rangos, campos obligatorios, valores por defecto) — igual de estricto que un formulario de la app

▼

4. Si los datos son válidos, se llama al MISMO motor de cálculo que usan las pantallas normales (`js/calc/*.js`) — es la fuente única de verdad, no hay una copia para la IA

▼

5. El resultado (números exactos) se guarda como una "corrida": queda en una tabla que se ve en pantalla y en el reporte, SIN pasar por el texto del modelo



6. El resultado vuelve al modelo, que lo interpreta, compara, y redacta la respuesta final en lenguaje natural (o pide otra herramienta más, si hace falta)

Si los datos que mandó el modelo no son válidos, la app no revienta: le devuelve un mensaje de error claro ("falta tal dato", "el rango permitido es de X a Y") y el modelo reintenta con datos corregidos — el usuario ni se entera de esos reintentos.

Las piezas (mapa de archivos)

Pieza	Qué hace
<code>js/calc/*.js</code>	Los motores de cálculo de ingeniería — los mismos de siempre, sin cambios para la IA.
<code>js/ai/tools.js</code>	El "menú" de herramientas: cada una declara qué datos necesita, con qué reglas, y a qué motor llama. Es el único lugar donde se conecta la IA con un cálculo.
<code>js/ai/analisis.js</code>	El bucle de la conversación: arma lo que se le manda al modelo (instrucciones del agente + historial + herramientas permitidas) y procesa lo que responde.
<code>js/ai/agentes-analisis.js</code>	Los agentes (perfiles): instrucciones de cada uno y qué herramientas tiene permitido usar.
<code>js/ai/reporte.js</code>	Arma las tablas de "Cálculos ejecutados" y el reporte final a partir de las corridas reales — nunca de lo que "dice" el modelo.

La garantía clave (por qué esto es confiable)

Que un número aparezca en una tabla del chat o del reporte **no depende de que el modelo lo escriba bien**: depende de que hubo una llamada real y válida a un motor de cálculo. Esto se refuerza en tres capas de código, no solo con instrucciones de texto:

1. Al modelo solo se le **describen** las herramientas que su agente tiene permitidas.
2. Aunque el modelo pida una herramienta no permitida, la app la **rechaza** antes de ejecutarla.
3. Las tablas de resultados se arman a partir de las corridas guardadas, **no** del texto que

redacta el modelo — así que aunque el modelo se equivoque redactando, la tabla no miente.

¿Qué es un "agente"?

Un agente = un nombre + instrucciones (cómo debe comportarse) + la lista de herramientas que puede usar. Hoy hay dos predefinidos en el Asistente técnico:

- **Estándar** — calcula rápido, usa valores por defecto razonables cuando falta un dato secundario y lo declara como supuesto.
- **Riguroso** — pensado para una memoria de cálculo formal y definitiva: pide y confirma

TODOS los datos antes de calcular (nunca asume en silencio), distingue si un valor lo dio el usuario, es un valor por defecto de la calculadora, o es una estimación de ingeniería con su justificación, y arma una memoria con estructura normativa (objeto, metodología, verificación de cumplimiento, conclusiones...).

También se pueden crear agentes propios (otro tono, otras instrucciones, otro subconjunto de herramientas habilitadas) desde la pestaña **Agentes** de cada pantalla de IA.

Para ir más a fondo

`docs/ia-herramientas.md` tiene el detalle completo: las 16 herramientas actuales una por una, cómo se valida cada dato, cómo agregar una herramienta nueva, y los límites conocidos del sistema.